

SDP RPV

Justin Cortes, Carson Quesada, Justin Ortiz, Jielsen Travis Cangco

December 2025

Advisor: Dr. Tamer Omar



CalPolyPomona

College of Engineering

1 Abstract

This project presents the development of a semi-autonomous ground vehicle that integrates modern sensing technologies, a redesigned computing architecture, and ROS2-based software to create a scalable research platform for autonomous navigation. The system incorporates a 2D LiDAR for obstacle detection, a 13-MP Orange Pi camera module for YOLO-based object recognition, a GPS module and IMU for global and local pose estimation, and a quadrature encoder for odometry. A major hardware transition from the Raspberry Pi 3 to the Orange Pi 5 enabled real-time execution of computationally intensive tasks, including LiDAR clustering, multi-sensor fusion, and neural network inference for object detection. The control pipeline was restructured to delegate motor actuation and telemetry to Arduino microcontrollers, improving timing determinism and system safety through features such as automatic emergency stop.

The implementation test bed demonstrates successful restoration and enhancement of legacy vehicle capabilities—including remote piloting, MJPEG video streaming, and a redesigned TCP protocol—alongside new autonomy-focused modules such as LiDAR object classification, safety-zone monitoring, YOLOv8 Nano detection, and a partial EKF-based localization stack. Initial results confirm reliable LiDAR integration, real-time visualization, accurate object clustering, and functional camera-based detection, establishing the platform as a robust foundation for continued autonomous system development. Although full GPS waypoint tracking and fused heading estimation remain untested, the project provides a validated proof of concept for a modernized autonomous vehicle test bed and offers a clear pathway toward future expansion of autonomous navigation capabilities.

Contents

1	Abstract	2
2	Introduction	4
3	Methodology	7
3.1	System Architecture	7
3.2	System Components description and functionalities	7
3.3	Software Migration to ROS	8
3.4	Restoring Legacy Features	9
4	Design Components	10
4.1	Yolo Camera	10
4.2	2D LiDAR	10
4.3	Orange Pi 5	11
5	Implementation Test Bed and Results	12
5.1	Object Detection	12
5.2	Camera/YOLO	15
5.3	Autonomous Mode	16
5.3.1	GPS	16
5.3.2	IMU	17
5.3.3	Odometry	17
5.3.4	Sensor Fusion	17
5.3.5	Heading Calibration	17
5.3.6	Pure Pursuit Algorithm	17
6	Conclusion	17
	References	19

2 Introduction

With the rapid improvements in technology during the 21st century, it is only natural that the most common mode of transportation is improved in significant ways. We are of course talking about motor vehicles and how in recent years they have pushed for full automation. Self-driving vehicles could revolutionize how we move from place to place.

Autonomous vehicles rely on a combination of advanced equipment, sophisticated algorithms, and real-time decision-making to navigate their environment safely and efficiently. The key components to achieve autonomy involve equipment used in AVs, global and local path planning, and perception systems that interpret the surroundings.[Synopsys, 2025]

Autonomous vehicles depend on a variety of sensors and hardware to perceive their environment and make decisions. These include LiDAR (Light Detection and Ranging) sensors, which create a 3D map of the surroundings, radar to detect objects at different distances, and cameras to detect objects and detect lanes. In addition, AVs use GPS and inertial measurement units (IMUs) for positioning and motion tracking. High-performance computing units and AI-driven software process these sensor data to make real-time decisions. Companies such as Tesla, Waymo, and Cruise utilize a combination of these technologies to enhance their self-driving systems. [Leiss, 2025]

Global path planning for autonomous vehicles is essential to determine the overall route an AV should take from its starting location to its final destination. This involves using high-definition maps, GPS data, and algorithms that consider road networks, traffic regulations, and potential obstacles. A well-optimized global path planning system ensures that the AV selects the safest and most efficient route while minimizing travel time. [Othman, 2022]

Perception is a critical component of AV systems as it allows the vehicle to detect and understand its surroundings. AV perception involves sensor fusion, where data from LiDAR, radar, and cameras are combined to create a comprehensive view of the environment. Machine learning algorithms process this information to recognize pedestrians, cyclists, traffic signals, and road signs.[Malik et al., 2022]

Local path planning focuses on the vehicle's immediate movement within its global path. It involves real-time decision making to handle dynamic elements such as pedestrians, traffic signals, and unexpected obstacles. This is achieved using trajectory optimization algorithms and real-time feedback from the AVs sensors. The system continuously adjusts the vehicle's speed and steering to navigate safely.

In terms of real-world applications, autonomous vehicle technology is already being tested and developed in various real-world scenarios. Companies like Waymo operate fully autonomous taxis in select cities, while Tesla's full self-driving system is available to consumers, gradually improving through over-the-air updates. In addition, autonomous trucks, such as those developed by TuSimple, are being tested for long-haul freight transportation, reducing the need for human drivers and improving efficiency. As these technologies continue to evolve, autonomous vehicles are expected to play an important role in the future of transportation, enhancing safety, reducing traffic congestion, and providing greater mobility for individuals who cannot drive.

As more and more autonomous vehicles are introduced on the roads they have the potential to drastically change many aspects of transportation. An advantage of using an infrastructure that supports autonomous vehicles is that AI could be used to eliminate human error. The use of AI could reduce accidents and drastically reduce traffic delays cause by human errors such as phantom traffic jams. Furthermore, some researchers say that autonomous vehicles could increase the amount of total miles traveled in a car due to the surge of new users who had not previously had access to a vehicle.[Bagloee et al., 2016] One of the biggest issues when moving to an infrastructure that supports autonomous vehicles is people that will continue to use regular cars because it is impossible to believe that everyone will be willing to switch to this new system. One possible solution for this problem is that we create a completely new lane that would only be for autonomous vehicles. This way the vehicles do not have to account for human error when moving around in crowded places such as the freeway. Though in theory this could work to be a solution to avoid fatal accidents, many will still be skeptical of an AV's safety.

Vehicle ownership is one possible area that can change with the uptake of autonomous vehicles. One possibility is that private vehicle ownership would decrease drastically as companies manage fleets of autonomous vehicles that individuals can request for transportation. If this were to happen, it would have several positive effects. Ride sharing would become more normal, decreasing the overall amount

of fuel used and reducing the amount of vehicles on the roads. The reduction of vehicles on the road would also mean more frequent usage and consequently shorter lifespans, which would allow the inclusion of newer and cleaner technologies quicker. It is also very reasonable to think that instead of fleets of autonomous vehicles owned by companies, these vehicles might be privately owned. Even with this paradigm, vehicle ownership per household has been predicted to decrease [Othman, 2022].

Vehicles make up a large contribution of greenhouse gas emissions. As was discussed, AVs would reduce the amount of cars on the road resulting in less vehicle emissions. AVs may also travel together in what are called platoons, where vehicles communicate with each other to help regulate speed. Better control over stopping and accelerating would improve fuel efficiency [Othman, 2022]. More on platoons is discussed in a future section.

Before an AV can even begin to make a decision, it must know its environment. AVs employ many sensors, each with their own strengths and weaknesses, to see their surroundings. The following is a list of equipment commonly used in AV technology. [Ignatious et al., 2022]

- **Cameras:** Cameras are the most common sensors used in AVs. This is because of the quality and quantity of information they are able to provide. Cameras are for classification and collecting information about shapes and textures. Because they can also take in images they are necessary for traffic sign recognition and distinguishing traffic light colors. They cannot however sense any depth information and suffer under poor lighting conditions and bad weather and so need to be paired with other sensors [Jebamikyous and Kashef, 2022].
- **LiDAR:** LiDAR is another important piece of equipment commonly used in AVs. This collects information about the environment by sending out light and seeing how long it takes for that light to travel back to the sensor. It can do this thousands of times a second. Doing so allows the vehicle to gain depth information and create what is called a point cloud map. Since it cannot read signs or see traffic light color it paired well with cameras [Jebamikyous and Kashef, 2022].
- **Radar:** Radar works similarly to LiDAR only instead of pulses of light radio signals being transmitted from an antenna. The distance from an object is determined by how long it takes for the radio signal to return to the antenna. Bad weather does not affect the radar’s ability to sense; however the drawback to radar is that the signals cannot form a detailed image of its environment [Jebamikyous and Kashef, 2022].

Once information about the surroundings are gathered an autonomous vehicle needs to be able understand it.

Recent advances in Lidar and camera feature fusion has proven successful in being able to detect objects on the road. average precision of 94.59KITTI dataset which is better than most of the previous methods. [Wu et al., 2023]

Semantic segmentation is the process of assigning every part of an image to a class. This would separate the roads, trees, vehicles and other objects in the picture into different classes. The information extracted from this technique helps AVs plan a route around their environment [Jebamikyous and Kashef, 2022].

Object detection is a crucial step in an AVs perception of its environment. Here an AV will identify objects and locate their position. Here the point cloud map comes into play. A point cloud map stores the position and color of many points. Different algorithms such as YOLOv2, YOLOv3 and Viola-Jones algorithm process this data to classify objects around the vehicle [Jebamikyous and Kashef, 2022].

In [Akhshirsh et al., 2021] a team designs a cost-effective solution for planning a path to a destination following that route. Their implementation used a GPS module to retrieve the robot’s current location, a digital compass to retrieve its bearing, and Google maps for planning its route[Sakkhor et al., 2015]. The robot first determines its position and from there it uses google maps to obtain waypoints along the route it plans. From there it performs calculations to obtain the desired heading and distance to the spot. Once it turns to its correct heading it then drives until it is within 1 meter of its destination. If so it has reached the point and stops. The team tested this design with a given set of points for the robot to follow. The robot followed the specified path with good accuracy, directing itself with a maximum error of about 2 meters from the next point. It occasionally ran into the issue of overturning or driving past its destination.

Another solution to path planning [Zein et al., 2018] is by first manually driving the vehicle and dropping waypoints. In doing so, the vehicle is creating the path in which it will follow and use GPS location to determine where to drop a waypoint when the wheels are turned a certain degree. After a path is successfully planned/created, the vehicle is now able to autonomously maneuver with little to no complications. It is important to note that although a path is set for the vehicle to follow, [Sun and Zhang, 2021] obstructions may still come into interference but because of a vehicle’s sensors, it will only delay the time of arrival. [GPR, Inc., 2024]

Once an AV has a clear image of its surroundings and a way point to move towards it can begin to safely navigate its environment. To do so there exist many algorithms based on different techniques.

Sampling based algorithms involve taking points in space and connecting these points to create a path from the start point to the destination point. A number of sampling based algorithms exist and some include rapidly exploring random trees, artificial potential fields and probabilistic road maps [Abdallaoui et al., 2022].

Node-based can be organized into 2 categories, grid based and graph based. Grid based algorithms involve splitting the image into cells with each cell representing an object. A path planning algorithm such as Dijkstra or A* is then used to plan a path. This algorithm is not good for fast moving obstacles however is a common method for slower paces. Graph based algorithms are very similar however allow for more complex relationships between nodes meaning they can be used around faster obstacles [Abdallaoui et al., 2022].

Mathematical based algorithms use geometrical approaches in order to model the environment and the system. These types of algorithms could further be split into two different subcategories. The first of these two categories is linear algorithms. This subcategory is used to control disturbances and uncertainties while path planing. Additionally, it this method plans its path by optimizing a path with the most flatness to ensure a smooth path. The second subcategory is optimal control uses two methods to plan a trajectory that avoids obstacles to create the most optimal path for the vehicle. [Abdallaoui et al., 2022]

Bio-inspired algorithms splits into two major algorithms. The first algorithm is evolutionary algorithms. Evolutionary algorithms use adaptive learning in order to find a optimal path for the vehicle to use. This type of algorithm uses four different methods to plan the path. These methods are genetic algorithm, particle swarm optimization, ant colony optimization, and shuffled frog leaping algorithm. For genetic algorithm the user would describe to environment to vehicle the using grids and then the vehicle would use two functions to plan its path. For particle swarm optimization they use particle randomly assigned a position and tell them to converge to a certain place. The algorithm will take the results from the particle and use the best particle path. Ant colony optimization is a method that combines elements from the previous two and simulates multiple possibilities in order to get the best results for the car. Finally shuffled frog leaping algorithm uses frog behavior while they are hunting for food in order to improves convergence speed, better search capacity with the existence of uncertainty. [Abdallaoui et al., 2022]

Neural network algorithms uses the systems that evolutionary algorithms use but in addition to that they also use additional methods to plan paths. These methods include fuzzy decision function, artificial neural network, Bayesian neural network, and deep reinforcement learning.

Each of these algorithms have their advantages and disadvantages which is what a multi-fusion-based algorithm aims to solve. A multi-fusion-based algorithm can use each algorithm where it is optimal making it good in both static and dynamic, fast paced environments. This type of path planning has its own drawbacks however. A multi-fusion-based algorithm sometimes will have each algorithm working separately which is not ideal. If the algorithms work together then there are situations, like emergency situations, where the uncertainties in the algorithm need to be properly controlled [Abdallaoui et al., 2022].

Even with so many developments in autonomous vehicle perception and decision making, there exist other methods to increase robustness and safety. One of these is V2X. V2X, or vehicle to everything, is a way for the vehicle to wirelessly communicate with other vehicles, infrastructure, pedestrians and more. Implementing such a strategy would have several benefits. Autonomous vehicles often run into problems in terms of their perception. Some scenarios include blindspots at intersections or poor weather conditions. Communication with other vehicles and infrastructure can compensate for these shortcomings by receiving information from many other sources to form a more accurate and complete picture of its environment. Communication with other vehicles can also improve efficiency and traffic control with

Cooperative adaptive cruise control, or CACC. CACC has a caravan of vehicles traveling together under one lead vehicle that sets the speed. This would help with fuel consumption and can help increase traffic safety [Jung et al., 2020].

To facilitate this communication vehicles can either directly communicate directly with other vehicles, infrastructure or other road users or they can share information through the use of network communication. For direct communication the ITS 5.9GHz frequency band has been reserved for vehicle communication. Communication through the network requires cellular connection and would give vehicles access to edge computing which can be used to process large amounts of data for vehicles [Tigadi and Chandra, 2023].

While there are many benefits to including this technology into autonomous vehicles it does come with its sets of challenges. Strong security would need to be put in place to prevent unauthorized access and privacy breaches. Careful policies and allocation of spectrum frequencies are also necessities that V2X will rely on as it becomes incorporated into vehicles [Tigadi and Chandra, 2023].

3 Methodology

3.1 System Architecture

Our system architecture removes some of the devices from the previous implementation. The Raspberry Pi 3 will be replaced as the chief processor and its role as TCP server and vehicle IO will be taken by the Orange pi 5 and a few arduinos. The Orange Pi now hosts the TCP server and will relay driving and lighting instructions to a set of arduinos. There will be three arduinos each with a separate role involving the vehicle's IO. One arduino controls the lighting subsystem on the vehicle, another controls the driving capabilities and the last one reads telemetry information and sends it to the Orange Pi when requested. lastly the camera and servo from previous groups is removed. The camera used in previous implementations was probably not compatible with the Orange Pi and if compatible would have been very hard to work worth or had poor performance. instead we use the official camera for the Orange Pi 5, the 13850 camera module.

In addition to being the TCP server and IO coordinator the Orange Pi will be the processor responsible for running the autopilot feature. To do this we added a WayPonDEV FHL-LD19, which is a 2D LiDAR for, localization and object detection. We also include the 13850 camera module for computer vision and a Teylten Robot BNO055 IMU for localization.

The decision to switch from the Raspberry Pi 3 to the Orange Pi 5 was made to accommodate for the large amount of processing that is required for the autonomous feature. Not only does the Orange Pi have a better processor for general processing but it includes an NPU that we can use for image processing. This will reduce the time it takes to process the camera data and reduce the power consumed by the vehicle giving us more battery life.

3.2 System Components description and functionalities

The purpose of the Orange Pi is to process the data from various sensors which will output high level autonomous driving commands. The pi will also receive V2X data to help make driving decisions. The data will be transmitted over wifi communication. This will simulate dedicated short range communication in autonomous vehicles (DSRC). In real world applications DSRC uses licensed radios, however we do not have access to that so communication over a wifi network was chosen. The pi will run a MQTT broker with Mosquitto to receive this communication. Though in real world applications ASN.1 or UPER is used the data will be sent and received in a JSON format. We will then try to incorporate this data into the decision making process for the vehicle.

The purpose of the 2D LiDAR sensor is to detect the surrounding environment of the vehicle. The 2D LiDAR will provide accurate position data for objects in the vehicle's surrounding environment. This will be used in obstacle detection and localization. To use this data for localization we will use a SLAM algorithm.

We will also implement a new camera to perceive its environment through different variables such as color that the 2D LiDAR sensor cannot. The camera will send data to the pi for processing. On the

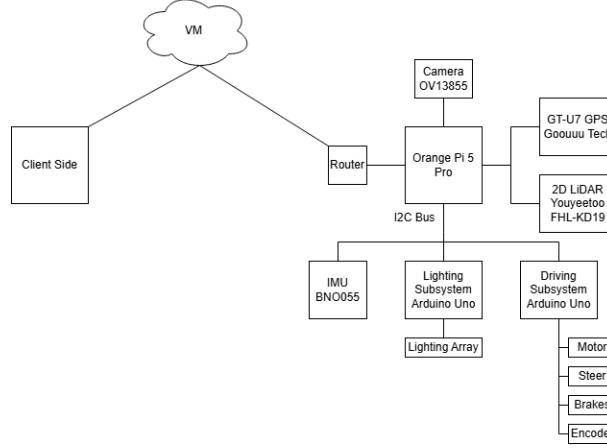


Figure 1: System Diagram

pi we will use computer vision algorithms like OpenCV and an algorithm called You Only Look Once (YOLO). We are thinking of using the YOLOv5s which should run well on the pi 5 we are planning on using. A visual SLAM algorithm will also be used for vehicle localization so that the vehicle can know exactly where it is in its environment.

The IMU will be utilized for robust localization. The IMU will feed data, along with all the data from the SLAM algorithms and GPS information, into the pi where the vehicle's location will be found. To incorporate all these inputs a Kalman filter will be used. The use of this filter will allow for a better estimate of where the vehicle's position currently is by combining predictions and real-world sensor data. Its key role will be beneficial in predicting the vehicles next position, as well as providing updates on estimates when GPS data is received. As a result, this would output a smooth and accurate trajectory of the vehicle despite any potential sensor errors.

To drive the vehicle the information from all devices will be combined together with sensor fusion and put through a pipeline to generate high level driving commands. This will be done by with a path planning algorithm, possibly the Dynamic Window Approach (DWA) or Model Predictive Control (MPC). This path created will be transformed into driving that will be sent to the existing pi that is used to control the vehicle.

3.3 Software Migration to ROS

The original system used a basic C++ program. To improve modularity and timing accuracy, we migrated to ROS2, an industry-standard robotics framework. ROS2 nodes now handle TCP communication, video streaming, and onboard device interfaces, making the system more maintainable and extensible.

The ROS implementation is defined by the set of nodes used. The system employs the following nodes:

- **TCP Delivery:** This node receives and sends TCP messages. It identifies each message type and routes the data to the appropriate processing module.
- **State/Mode:** This node maintains the current state or mode of the vehicle. The vehicle can operate in several modes: manual mode, arming mode (which prepares the system for autopilot), autopilot mode, emergency stop, and fault mode for when the vehicle encounters an issue.
- **Drive Arbiter:** The drive arbiter receives drive commands and applies the required logic to them. Autopilot can request control from the drive arbiter and take authority over the vehicle. When autopilot has internal control, any incoming manual drive command overrides it. The drive arbiter also enforces maximum speed limits and triggers emergency braking when an object is detected.

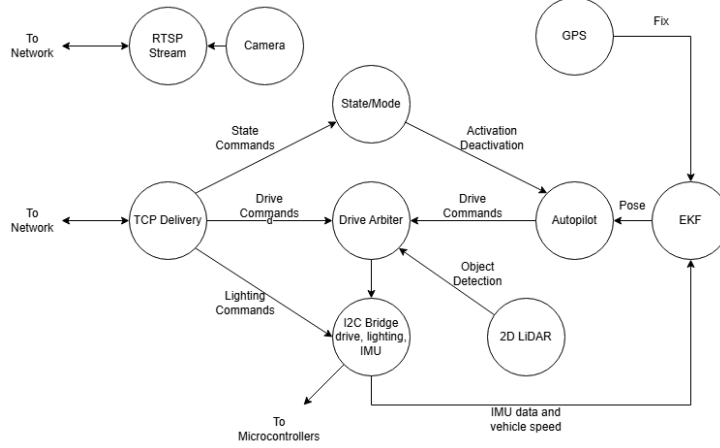


Figure 2: GUI for the client-side application

- **I2C Bridge:** This node facilitates communication with devices on the Orange Pi’s I2C bus. It delivers drive commands to the drive subsystem and receives speed and steering measurements. It also delivers lighting commands to the lighting subsystem and receives IMU data.
- **2D LiDAR:** This node receives and processes LiDAR data. It publishes object detections used by the drive arbiter.
- **GPS:** The GPS node receives data from the GPS device and publishes it for use by the extended Kalman filter (EKF).
- **EKF:** The EKF node is a built-in ROS2 component implementing an extended Kalman filter. It fuses telemetry data to estimate an accurate vehicle position and orientation.
- **Autopilot:** The autopilot node receives the filtered pose from the EKF and applies a pure pursuit controller to follow a predefined set of GPS waypoints.
- **Camera:** The camera node acquires video frames from the onboard camera and publishes them.
- **RTSP Streamer:** This node receives raw images published by the camera node and transmits them via RTSP.

The diagram provided to illustrate the ROS structure is not comprehensive. Data flows in many directions between many nodes, and the displayed connections are intended only to give an overview of how the components interact.

3.4 Restoring Legacy Features

Introducing a hardware switch required restoring several legacy features of the vehicle. This process involved rewiring components, installing tools, and updating software. While doing so, we also improved several systems to make the platform more scalable and manageable as the project grows.

To restore remote control, a cloud-based virtual machine (VM) was configured to forward network traffic. The previous implementation used OpenVPN; the new setup uses Fast Reverse Proxy (FRP) to relay both control messages and the video feed. Video is streamed over RTSP rather than WebRTC. Although this setup does not deliver video as efficiently, this approach is arguably better for our purpose. Because the project is no longer a remote piloted vehicle a simpler setup is more ideal. It also integrates nicely with the GUI developed as a tool for operating the vehicle.

The TCP protocol was redesigned to support bidirectional communication. Previously, the vehicle only received fixed-length command strings. The updated protocol sends variable-length messages with

headers specifying size and type, allowing the client to receive vehicle status updates and making it easier to add new message types.

The Windows-based client application was significantly improved. The original program offered minimal output, cumbersome connection handling, and required manually opening a browser to view the camera feed. The updated version incorporates a graphical user interface (GUI) built with Dear ImGui. Users can now connect or disconnect with a button, view real-time vehicle status, and see the video feed directly within the application. Dependencies include Dear ImGui, stb_image for texture rendering, GLFW for window and input handling, GLAD for OpenGL loading, and FFmpeg for retrieving the RTSP video stream.

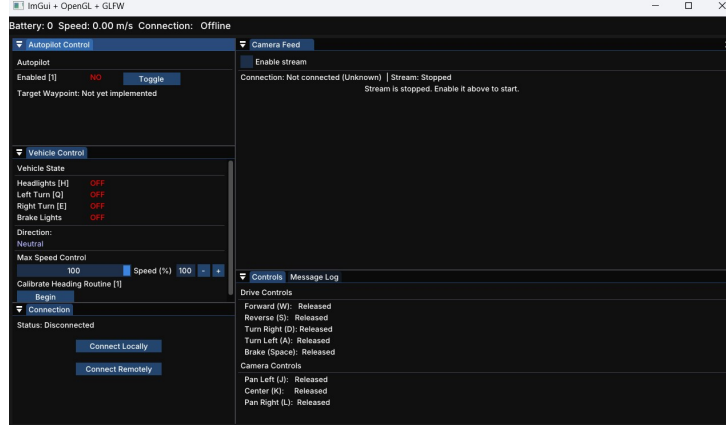


Figure 3: GUI for the client-side application

4 Design Components

4.1 Yolo Camera

The Orange Pi 5 Camera Module 138550 is the camera that we decided to use for the object detection and recognition. The camera is 13 MP and which provides a maximum resolution of 4224x3136. The Orange Pi camera captures picture and could also record digital videos. For the project we used these capabilities to allow the car to scan the environment in front of it. Additionally we used it to get a live feed of the car as we control it or as it is in driving around

4.2 2D LiDAR

The **Youyeetoo FHL-LD19** LiDAR sensor model was the chosen component to be used for object detection due to its key features that can be used for robotics and autonomous applications. The specifications would include:

- **Type:** 2D LiDAR (Light Detection and Ranging)
- **Scan Range:** Up to 12 meters
- **Accuracy:** ± 45 mm (average error)
- **Angular Resolution:** $\sim 0.5-1^\circ$
- **Scanning Frequency:** 5-13Hz (externally controlled by PWM)
- **Measuring Frequency:** 4500Hz (varies by model)
- **Field of View:** 360° full scan

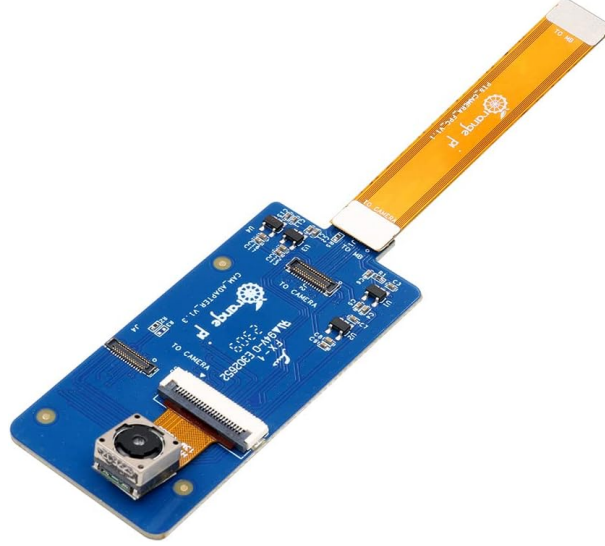


Figure 4: Orange Pi Camera

- **Interface:** UART/Serial (TTL) communication
- **Power:** Usually 5V DC

The 2D LiDAR provides a real-time 360° environmental scanning with $\pm 45\text{mm}$ accuracy. Its 12-meter range and 5-13 Hz refresh rate enable reliable detection of stationary and moving obstacles, while the $0.5\text{-}1^\circ$ angular resolution allows distinction between closely spaced objects. Despite limitations in vertical detection and sensitivity to highly reflective surfaces, the LD19 offers an optimal balance of performance, power efficiency, and cost for our applications.

4.3 Orange Pi 5

The Orange Pi 5 serves as the main onboard computer for the vehicle and is responsible for processing all high-level perception and navigation tasks. Its upgraded CPU and integrated Neural Processing Unit (NPU) provide the performance necessary to run computationally intensive algorithms such as LiDAR processing, YOLO-based object detection, and multi-sensor fusion. This represents a significant improvement over the previous Raspberry Pi 3, which lacked the resources required for real-time autonomous operation.

In the hardware system, the Orange Pi 5 functions as the central coordinator between sensors and microcontrollers. It receives data from the 2D LiDAR, camera, GPS module, and IMU, and communicates with the Arduino-based motor controller through I2C to issue steering and throttle commands. This division of responsibilities allows the Orange Pi to focus on decision-making while the Arduino handles timing-critical motor control.

The Orange Pi also hosts the system's communication interfaces, including TCP networking for remote piloting and video streaming. Its compatibility with the Orange Pi Camera Module 13850 ensures reliable image capture for object detection and environmental awareness.

Overall, the Orange Pi 5 provides the computational capability, connectivity, and sensor integration required to support the autonomous features of the vehicle, making it the central hardware platform for the system's perception and control pipeline.



Figure 5: FHL-LD19 model



Figure 6: Orange Pi 5

5 Implementation Test Bed and Results

5.1 Object Detection

Initial verification and setup of the object detection pipeline began with hardware integration, which involved installing and configuring correct device drivers. This was done to ensure the 2D LiDAR sensor could stream the data to the ROS 2 network. In order to validate this data flow, a web viewer was created that displayed real-time LiDAR operation without needing desktop ROS tools. The images below showcase this successful integration by depicting raw LiDAR points, along with distance measurements. This initial setup phase confirmed the sensor’s field of view, range accuracy, and data publication rate.

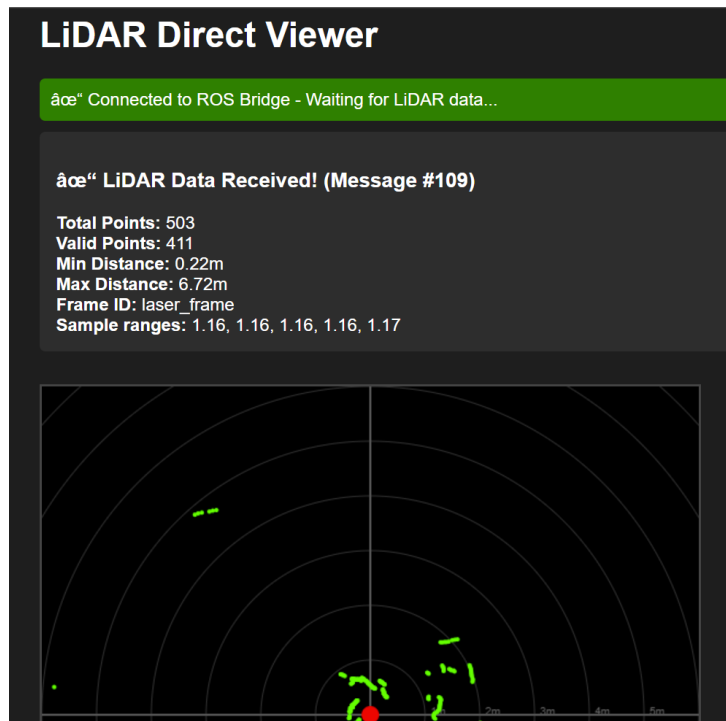


Figure 7: Raw LiDAR data visualization

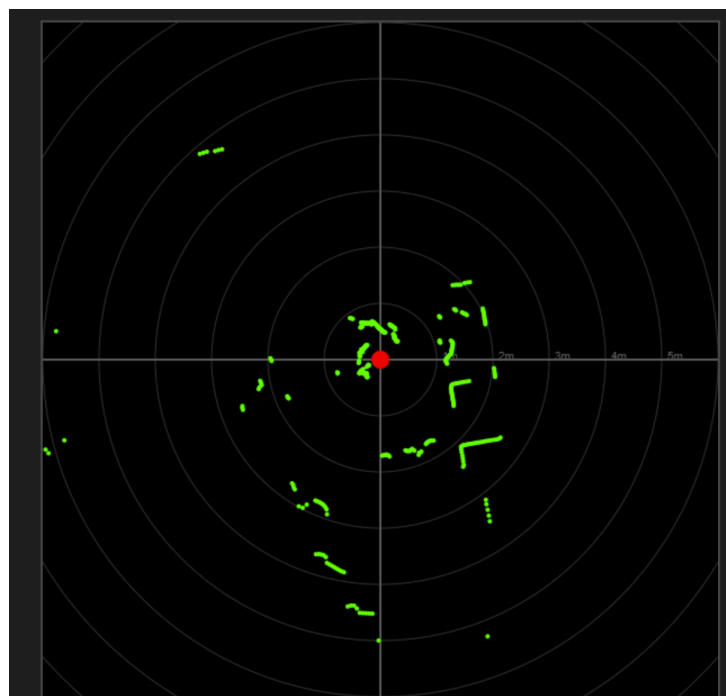


Figure 8: Raw LiDAR data visualization

The figure below was taken from the same web viewer, but now shows the output of the object detection node. The image now has the LiDAR points annotated with color shapes and text labels, such as “small_obstacle” that are being generated by the `publish_markers()` function. The visual output also confirms that the `classify_object()` function can categorize detected clusters based on their spatial properties, fulfilling one of the system’s core purposes of environmental perception.

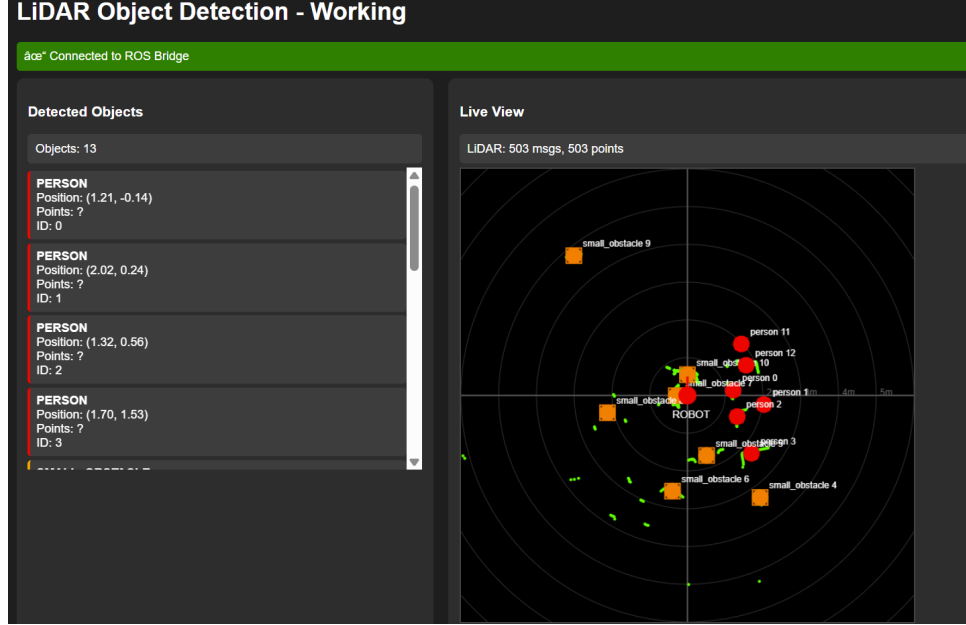


Figure 9: LiDAR visualization with detected objects labeled

Beyond visual identification, the object detection system provides a critical safety function by integrating real-time decision logic that utilizes distance-based zones for emergency braking. The LiDAR-based system implements three safety thresholds which involve, “Emergency”, “Slow-down”, and “Warning” zones. At the core of the emergency decision logic is a multi-layer detection strategy implemented through the `check_emergency_braking_with_objects()` function where the LiDAR determines whether an emergency stop is needed that is based on a combined analysis of angle, distance, and object type. Only objects within the central ± 20 degrees emergency angle zone can trigger immediate full braking when within the emergency distance. Objects detected in the vehicle’s peripheral are considered to be in the caution zone, in which the vehicle’s speed is reduced. This prevents unnecessary emergency stops for side obstacles while maintaining aggressive protection against direct frontal collisions.

Outside testing of the program was able to correctly configure objects in the vehicle’s path. In fact, the image provided showcases the area in which object detection is taken into account to determine when to slow down and initiate an emergency stop. It is important to note that the speed of the vehicle is not taken into account, which may lead to close calls and last minute braking. As a temporary fix, increasing the “Emergency” and “Slow-Down” distance is a viable option but collaboration with the vehicle’s speed node may result in further improving the brake measurement.

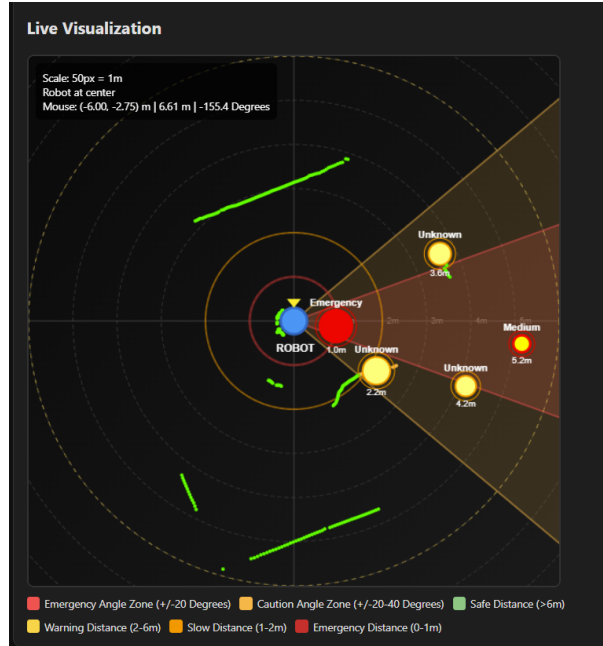


Figure 10: Emergency and Slow-down Zones

5.2 Camera/YOLO

In order for the car to detect objects we programmed the orange pi to support a camera. The camera was then programmed to support YOLOV8 NANO. Initially the camera was programmed to capture a single frame and then identify the objects in the output frame. The frame was then made a temporary file that could be viewed within vscode. Below is a frame from initial testing from when the YOLO program was first implemented.

After verifying that camera could run YOLO we decided to move on from just a single frame to the camera capturing a video. Since other coding editors besides vscode might be used in the future we decided to instead output the results into a web browser instead of making a temporary file in vscode. This new code allowed for a continuous stream from the camera while the vehicle was being piloted and in the autonomous mode. The code also narrowed the type of things that yolo could identify so only things relevant to a vehicle would be detected. Somethings that can be identified are people, vehicles, bikes, traffic lights, and backpacks.

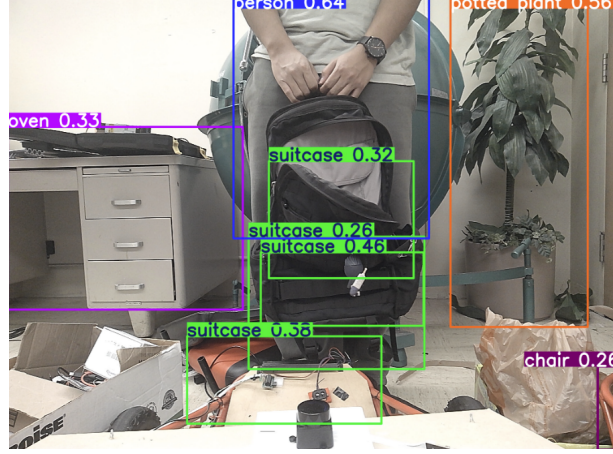


Figure 11: Yolo identification of multiple objects

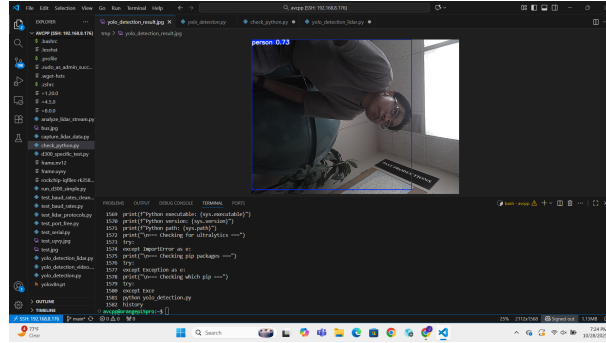


Figure 12: YOLO identification of a person

In order to get the camera feed from the car while it was being operated we created an additional node for a webviewer that would post the annotated images and camera detections to a separate tab. This feature did not end up functioning properly. Sometimes the annotated images would be published to the webviewer properly but others there would be errors that would show and the images would not be published. We ran out of time before we figured out what was causing this issue.

Through out the testing some object were misidentified which could possibly have to do with the resolution of the camera. This is displayed in Figure 11 where many of the objects are misidentified such as the backpack and the desk. There was an attempt to fuse the LiDAR and the camera so that the car would get additional details about the objects being detected such as height and distance from the car. We were unable to fully implement the code into the car.

5.3 Autonomous Mode

An initial attempt was made to develop an autonomous driving mode in which the vehicle could follow a sequence of GPS waypoints. Achieving this required a reliable estimate of global position and heading, which in turn depended on multiple sensors and sensor fusion. This section outlines the components used and the methods employed.

5.3.1 GPS

Global position was obtained using a GT-U7 GPS module based on the u-blox NEO-7 chipset. The module communicated with the Orange Pi over UART and provided position, time, fix quality, satellite

count, accuracy estimates, speed, and heading at 5 Hz. Although suitable for long-distance navigation, its reported positional accuracy of 1–5 m was insufficient for precise waypoint tracking over short distances.

5.3.2 IMU

Because GPS alone is noisy and prone to drift, an IMU was used to supply continuous corrections. The selected sensor was Teyliten’s BNO055, which reports orientation, angular velocity, linear acceleration, magnetic field measurements, and calibration status over I²C. Ideally, the magnetometer would provide absolute heading by referencing Earth’s magnetic field. However, the sensor’s placement near motors and other electronics introduced significant magnetic interference, preventing reliable magnetometer readings and limiting the IMU’s ability to determine absolute orientation.

5.3.3 Odometry

Short-term velocity estimates were provided by the quadrature encoder used by the Arduino motor controller. The Arduino forwarded speed data to the Orange Pi, allowing odometry to complement GPS and IMU measurements, especially over short time intervals.

5.3.4 Sensor Fusion

Sensor data were combined using ROS2’s built-in Extended Kalman Filter (EKF). Prior to fusion, raw GPS coordinates were converted from latitude/longitude to Cartesian x - y coordinates using the `navsat.transform` node. The EKF then fused GPS, IMU, and odometry inputs to estimate the vehicle’s pose. In principle, GPS provides global reference information to correct long-term drift, while the IMU and odometry reduce short-term error and improve responsiveness. Although this pipeline was implemented, the fused output—particularly heading—remains untested due to time constraints.

5.3.5 Heading Calibration

Because the magnetometer could not be used, a separate heading initialization procedure was implemented. The vehicle first averages several stationary GPS readings to estimate its initial position, then drives forward approximately 10 m and records a second position estimate. The heading is computed from the displacement vector between the two points. This method was implemented but also remains untested.

5.3.6 Pure Pursuit Algorithm

Assuming accurate pose estimates, the vehicle uses a pure pursuit controller to track the GPS waypoint path. This algorithm was selected for its simplicity and suitability for low-speed platforms. However, due to limited time, the controller could not be tested on the vehicle.

6 Conclusion

This project demonstrates a substantial advancement in the development of a semi-autonomous ground vehicle by integrating modern sensing technologies, a redesigned computing architecture, and a scalable ROS2-based software framework. Through the transition from the Raspberry Pi 3 to the more capable Orange Pi 5, the system successfully achieved real-time execution of computationally intensive tasks, including LiDAR clustering, YOLO-based object detection, multi-sensor data streaming, and high-level decision-making. The redesigned architecture—supported by modular ROS2 nodes, microcontroller-based motor control, and a bidirectional TCP communication protocol—proved to be both robust and flexible, enabling streamlined debugging, improved maintainability, and future extensibility.

Experimental validation confirmed correct operation of key subsystems such as LiDAR-based object classification, safety-zone detection logic, real-time camera inference, MJPEG video streaming, and telemetry exchange across the system’s distributed architecture. These capabilities establish a strong

foundation for autonomous navigation and demonstrate the feasibility of merging perception, localization, and control within a compact research platform.

Despite these achievements, several challenges remain before fully autonomous operation can be realized. The fused pose estimation pipeline, GPS waypoint tracking, and heading calibration procedures require on-vehicle testing to validate long-term stability and control accuracy. Limitations such as GPS positional drift, magnetic interference affecting IMU heading, and occasional YOLO misclassifications highlight the need for improved sensor fusion strategies and more rigorous calibration. Furthermore, full integration of LiDAR-camera fusion for enhanced semantic perception remains an important future milestone.

Overall, this work provides a successful modernization of the vehicle’s hardware and software infrastructure while establishing a clear pathway toward reliable autonomous navigation. The results confirm the viability of the proposed multi-sensor, ROS2-driven architecture and position the platform as a capable and extensible test bed for future research in autonomous systems, perception pipelines, and intelligent vehicle control.

References

- [Abdallaoui et al., 2022] Abdallaoui, S., Aglzim, E.-H., Chaibet, A., and Kribèche, A. (2022). Thorough review analysis of safe control of autonomous vehicles: Path planning and navigation techniques. *Energies*, 15(4).
- [Akhshirsh et al., 2021] Akhshirsh, G., Al-Salihi, N., and Hamid, O. H. (2021). A cost-effective gps-aided autonomous guided vehicle for global path planning. *Bulletin of Electrical Engineering and Informatics*, 10(2):650–657.
- [Bagloee et al., 2016] Bagloee, S. A., Tavana, M., Asadi, M., and Oliver, T. (2016). Autonomous vehicles: challenges, opportunities, and future implications for transportation policies. *Journal of modern transportation*, 24:284–303.
- [GPR, Inc., 2024] GPR, Inc. (2024). The 5 essential building blocks for driverless navigation. *GPR*.
- [Ignatious et al., 2022] Ignatious, H. A., Hesham-El-Sayed, and Khan, M. (2022). An overview of sensors in autonomous vehicles. *Procedia Computer Science*, 198:736–741. 12th International Conference on Emerging Ubiquitous Systems and Pervasive Networks / 11th International Conference on Current and Future Trends of Information and Communication Technologies in Healthcare.
- [Jebamikyous and Kashef, 2022] Jebamikyous, H.-H. and Kashef, R. (2022). Autonomous vehicles perception (avp) using deep learning: Modeling, assessment, and challenges. *IEEE Access*, 10:10523–10535.
- [Jung et al., 2020] Jung, C., Lee, D., Lee, S., and Shim, D. H. (2020). V2x-communication-aided autonomous driving: System design and experimental validation. *Sensors*, 20(10).
- [Leiss, 2025] Leiss, P. J. (2025). The functional components of autonomous vehicles. *Robson Forensic*.
- [Malik et al., 2022] Malik, S., Khan, M. A., El-Sayed, H., Khan, J., and Ullah, O. (2022). How do autonomous vehicles decide? *Sensors*, 23(1):317.
- [Othman, 2022] Othman, K. (2022). Exploring the implications of autonomous vehicles: A comprehensive review. *Innovative Infrastructure Solutions*, 7(2):165.
- [Sakthor et al., 2015] Sakthor, T. H., Zaman, S., Sadi, M. A. A., and Mahmud, A. A. (2015). Autonomous car using full mapping gps system.
- [Sun and Zhang, 2021] Sun, S. and Zhang, Y. D. (2021). 4d automotive radar sensing for autonomous vehicles: A sparsity-oriented approach. *IEEE Journal of Selected Topics in Signal Processing*, 15(4):879–891.
- [Synopsys, 2025] Synopsys (2025). What is an autonomous car? - how self-driving cars work. *Synopsis Glossary*.
- [Tigadi and Chandra, 2023] Tigadi, A. and Chandra, M. (2023). Literature review on the future of v2x communication in connected and autonomous vehicles. *International Journal of Research and Analytical Reviews*, 10:5.
- [Wu et al., 2023] Wu, Q., Li, X., Wang, K., and Bilal, H. (2023). Regional feature fusion for on-road detection of objects using camera and 3d-lidar in high-speed autonomous vehicles. *Soft Computing*, 27(23):18195–18213.
- [Zein et al., 2018] Zein, Y., Darwiche, M., and Mokhiamar, O. (2018). Gps tracking system for autonomous vehicles. *Alexandria Engineering Journal*, 57(4):3127–3137.